# 2 Architecture patterns

Performed by *Nikita Niakhai*

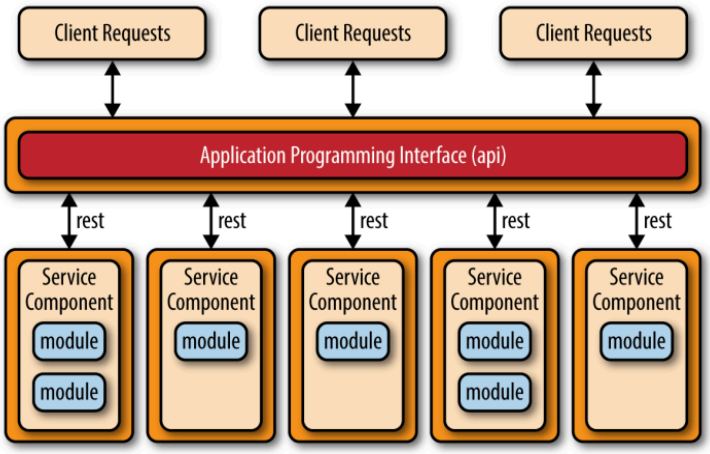
## 1.1

software architecture - is a skeleton, that provides methods, concepts (models) and vision, how to design a software piece. relationships (processes and their interactions) inside and outside such a software are described in an architecture (how to deploy in hardware). it consists of behavioural and dynamic descriptions of a system.

software architecture - is an abstraction that consists of patterns how to create a software piece: layered, microservices, microkernel, event-drive, API REST, Application REST and etc.

software architecture - a particular way to address non-functional requirements and quality attributes that used to describe a system

## 1.2.




**API Rest topology**

Client sends a request(s) that corresponds to certain querry or business logic, that is implemented in a server (back-end).

Back-end is written in e.g. MVC, so it provides many services that corresponds and awoke by different client requests.

A lot of modern apps in web and in application use such topology. It is implemented in various stacks of technologies and in many hardware pieces. From your e-commerce to some Arduino display in the gas-station.

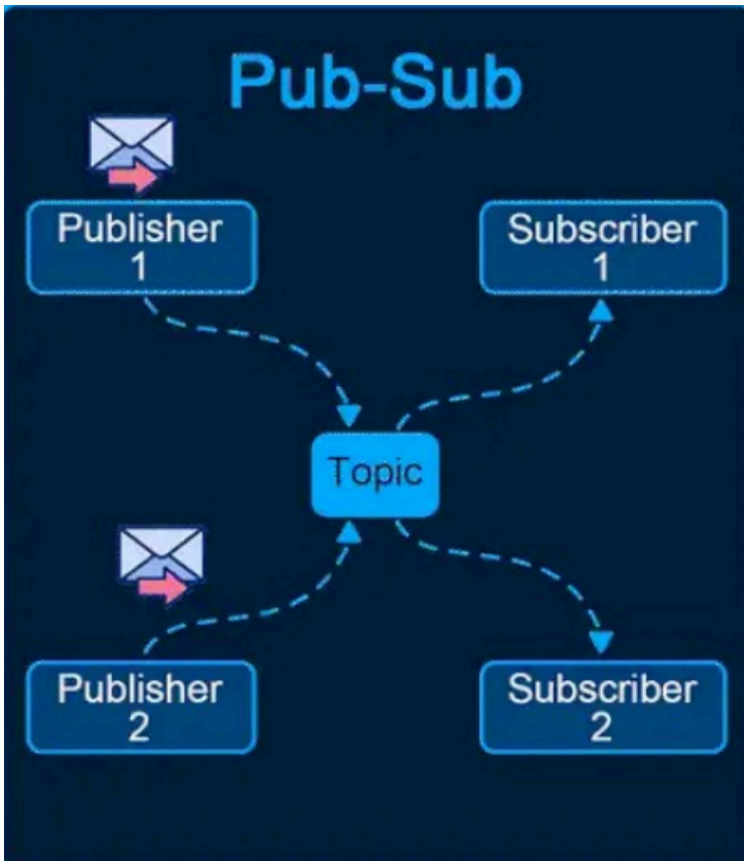
REST topology is a way of communicating, so in order to have proper “partner”, you need to authenticate a connection.

**Request-Response**


Request may be one of HTTP methods for example, PUT, DELETE.

Usually there are 3 components: Client-Server-DB. Client sends request via HTTP to Server. Server processes it and then talks to DB via some Queries (depends on framework and libs), then marshalls the data and replies to a Client.

**Publish-Subscribe**




Well explained with example of NEWS (magazines, papers) services. There is a magazine a New York Time, that has editors, journalists - creators. They create data (messages), There are readers of the magazine - subscribers. They want to get access to data in different ways stream, non-stop, or for a short period of time, but in all cases, they need to subscribe to a magazine. So they do it and after that magazine gives them access to data that was generated via publishers.

This topology may be formed in different ways and can turn more complex.

Some server-client communication, realized in back-end via Java or Python socket, uses such a topology.

Also uses concept of broker (middleware).

# 1.3

System architecture - is a scheme/description of a whole system that consists of software, hardware, devices, routers, sides.

Software architecture - is a scheme/description of a system that consists of  components, modules, services, interfaces, data.

They have different view and scope, so that a bit topologies. Sometimes identical qualities/requirements.

system architecture example: an online bank decides on regions, active-active data centers, external KYC provider, internal ledger service, call-center workflow, fraud engine, and reporting lake. It defines contracts, SLAs, and network zones across all of them.

software architecture example: the ledger service is split into write/read models, exposes gRPC for internal calls and Kafka events, stores money movements in an append-only event store, implements idempotency and exactly-once semantics, and provides domain modules with anti-corruption layers.

# 1.4

**Software correctness** means the system returns exactly the right outputs for all inputs and conditions defined in the spec, for example, a tax calculator computes the exact due amount for every documented tax scenario. **Software robustness** means the system still behaves reasonably when faced with invalid, unexpected, or failing conditions.

A pub/sub broadcasts each message to all subscribers so everyone gets a copy. A **point-to-point** load-balances messages across consumers so only one worker handles each message.

Architecture covers the high-level structure and cross-cutting decisions like component boundaries, protocols, and quality attributes; for example, choosing microservices with gRPC, event sourcing, and zero-trust networking. Design focuses on the detailed choices inside components—classes, patterns, data structures, and algorithms.

A user interacts directly with the system's UI or API, for example, a buyer placing an order on the website. A **primary stakeholder** has major goals or accountability tied to the system's success, e.g. director of E-commerce owning conversion KPIs. **A secondary stakeholder** is indirectly affected or provides support with less direct influence, for example, finance analysts consuming monthly sales exports.

**Cohesion** is how tightly a module focuses on a single, well-defined purpose (higher is better). **Coupling** is the degree of interdependence between modules (lower is better).

# 2.1

The main 4 are listed below. Also OOP uses: classes, object, interfaces, abstract classes, access modifiers (private, public, protected), method overriding, constructors. I am not digging into

explaining all of these, if it is necessary to achieve better grade I can talk about these in private. So now brief explanation of main 4.

Abstraction - Provide a simple interface, hide implementation details/.
Give people a simple handle and hide the wiring—client sees “what,” not the “how,” like a `earnMoney()` that returns `{ currency, amount}` without exposing headers, retries, or error parsing.

Encapsulation - keep state private, mutate via methods (e.g., controller of TV with buttons ).
Keeps data locked up and only let it change through approved way —e.g., a `Wallet` keeps a private items ( `dollars, coins, bitcoins` ) array and exposes just `addMoney()` and `total()`.

Polymorphism - One interface, many behaviors
Different shapes, same handshake—multiple objects share one interface but act differently, like `render()` on `Button` , `Link` , and `IconButton` producing different DOM.

Inheritance - Reuse/extend a base class (son and father example) IS-A.

e.g. `AdminUser` extends `User` , keeps the logic that was implemented in `User` , and adds more functions, also there is ability to change logic of already implemted functions.

# 2.2

This is a scope of system description that highlights and focuses on layered architecture, distribution, networking.

This is a structured abstraction of a system's design that simplifies complexity and highlights key characteristics for specific stakeholders, focusing on particular "concerns" like structure, behavior, or data.

1. **Performance:** The system’s ability to respond quickly and efficiently under varying loads, measured by latency, throughput, or response time.
2. **Scalability:** The system’s capacity to grow or adapt to increasing demands without significant performance degradation or manual intervention.
3. **Reliability:** The ability of the system to operate consistently and without failure over time, even in the face of unexpected conditions.
4. **Availability:** The percentage of time the system is operational and accessible, ensuring minimal downtime or interruptions.
5. **Security:** The system’s capacity to protect against unauthorized access, data breaches, and other security threats, ensuring integrity and confidentiality.

6. **Usability:** The system’s user-friendliness, ensuring that it is intuitive and accessible for its intended audience.
7. **Compatibility:** The system's ability to function and integrate with other systems, tools, or standards it interacts with.
8. **Maintainability**: Ease of ongoing updates and fixes?
9. **Testability**: How easily the system can be tested?
10. **Portability**: How well the system works across platforms?
11. **Interoperability**: Ability to integrate with other systems?
12. **Extensibility**: How easy it is to add new features?

**Microservices Architecture Pattern**

- **Problem:** Scalability and maintainability challenges in monolithic architectures.
- **Context:** Systems with multiple business domains requiring independent scalability and deployment.
- **Solution:** Break down the system into smaller, independently deployable services, each managing its domain.
- **Forces:** Balances **independence** and **decoupling** against **distributed systems complexity** and **communication overhead**.
- **Quality Impact:** Improves **scalability**, **deployability**, and **modifiability**, but might reduce **data consistency** and **increase complexity**.
- **Trade-offs:** Increased **operational overhead** (e.g., service discovery, load balancing, monitoring).
- **Known Uses:** Used by companies like **Netflix**, **Uber**, and **Amazon**.
- **Implementation Notes:** Use **Docker**, **Kubernetes**, **API Gateway** for service orchestration and communication.

# 2.3

**UML** (unified modelling language) - diagrams of 2 types behaviour and structural: use-cases, classes, state and etc. Used in system and business analysis

**ADR** - architectural design record. We create files in specific format that describes *what* was chosen (tech decision, stack) and *why.* There are many methodologies how to write ADR, that depend on scope and goals of the project.

**Patterns**

simplify future updates, reducing technical debt and enabling
seamless integration of new features and provide structured frameworks that help in building resilient
and secure systems from the ground up.

**Layers and tiers approaches.**

The architecture of a system is generally achieved by decomposing it into subsystems, following a layered and/or a partition based approach
These are orthogonal approaches:

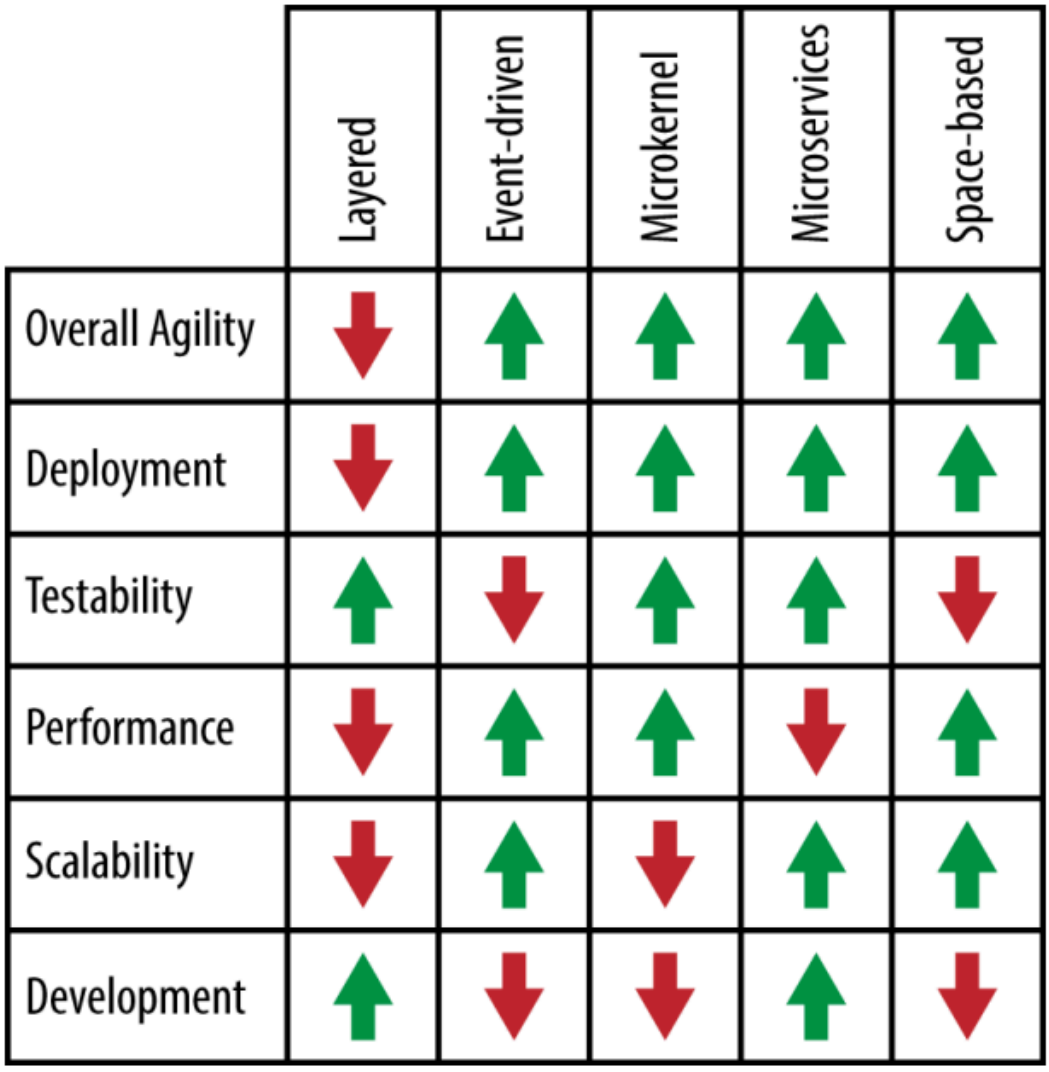
**a layer (onion)** is a logical structuring mechanism for the elements, that make up a software solution (e.g., a kernel is a layer)
**a tier (garlic)** is a physical structuring mechanism for the system infrastructure (e.g., a user interface is a tier). Tiered, Multitiered (1, 2, 3 - modern ones)

A 1-tier model (monolithic) describes a single-tiered application in which the user interface and data access code are combined into a single program from a single platform
A 2-tiers model (client/server) represents a split monolithic model composed by a client tier that interacts directly with a server tier
A 3-tiers model (n-tiers) is a client/server model in which the presentation, the application processing, and the data management are logically (ad often physically) separate processes

**ISO/OSI** reference model defines 7 network layers, characterized by an increasing level of abstraction.

**A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing - memo technique to memorize layers

Pattern analysis approaches - Comparison




## 2.4.

| Approach | Purpose / Scope | Strengths | Limitations | When to use |
|---|---|---|---|---|
| **UML** | Visual modelling of **structure** and **behaviour** for software & systems; used in system/business analysis. | Standard notation; good for communicating complex structure/flows; tool support.<br><br>Understandable for | Can get heavy/noisy; risks “diagram-driven” analysis without executable truth.<br><br>You need to use a lot of models to describe a system. | You need shared visuals across teams, formal reviews, or to clarify domain/runtime interactions. |

| Approach | Purpose / Scope | Strengths | Limitations | When to use |
|---|---|---|---|---|
| | | every team member even non-tech. | | |
| **ADR** | Capture what decision you made and why (context, alternatives, consequences). | Documents rationale.<br><br>Easy to version.<br><br>Reduces tribal knowledge. | Not a design picture;<br><br>Can go stale without governance.<br><br>It just used to describe whys not a whole picture of a system. | Any non-trivial tech/arch choice (protocols, DBs, patterns, vendors). |
| **Architecture Patterns** | Reusable solutions to recurring structural/behavioural problems with known trade-offs.<br><br>Guide system qualities. | Accelerate design. simplify future updates, reduce tech debt, enable resilient/secure foundations. | “No silver bullets”.<br><br>Complexity/consistency.<br><br>Need fit to context. | You want structured frameworks that target quality attributes (scalability, modifiability, reliability). |
| **Layers vs. Tiers** | Layers (onion): logical separation of concerns.<br><br>Tiers (garlic): physical/runtime separation across nodes. | Clear responsibilities (layers).<br><br>Deployability/scaling (tiers). I<br><br>f combine both - amazing. | Too many layers→latency/over-engineering.<br><br>Too many tiers→ops complexity. | You’re decomposing systems: decide responsibilities (layers) and physical deployment (tiers). |
| **ISO/OSI** | Networking reference architecture: 7 layers, increasing abstraction. | Shared vocabulary; isolates concerns; maps protocols to layers. | Reference, not an implementation plan.<br><br>Modern stacks blur layers. | Education, troubleshooting, protocol placement, threat modelling at network seams. |

# Part B [Team work]

GROUP 2
> NIKITA NIAKHAI
> ROMAN SOLOVEV
> VICTOR ADEKANYE
> SALEKH MAMADALIEV

# Task 3

## 3.1 Requirements:

### a. Functional Requirements:

1. The Application should be able to accept and process stateless Data stream from sets of registered IOT devices.
2. In real time, the Application would format the data into a stunning visual dashboard.
3. The Application should execute Predefined Queries
4. IS3 would be able to Authorize and Authenticate Users access.

### b. Non-Functional Requirements

1. Usability for User Experience. The front-end must be user-friendly, with minimal learning curve, and optimized for both web and mobile (iOS and Android).
2. Extensibility/Flexibility. Allow easy integration of new IoT devices, additional features, or external services without significant changes to the core system.
3. Interoperability. Support a variety of IoT devices and standards (e.g., MQTT protocol for device communication).
4. Scalability. Capability of handling an increasing number of devices and users without performance degradation.
5. Availability. The system should ensure 99.99% uptime through high availability architectures (e.g., multi-AZ deployments, failover mechanisms).
6. Reliability & Disaster Recovery. Ability to handle device communication failures gracefully and provide fallback mechanisms (e.g., retries, logging of errors). Also for the architecture it is obligatory to have a robust disaster recovery plan, including automated backups, replication, and quick failover.
7. Data Consistency & Integrity. The system should maintain consistency between the relational metadata and timeseries data, particularly when scaling or in case of a service failure. Also It ensures that the data received from IoT devices is stored correctly in the database without loss or corruption (good serialisation and marshalling).
8. Maintainability of documentation. Well-documented APIs, clear code standards, and centralized logging (ELK stack) for easy debugging and maintenance.

## 3.2 Components mapping and outline

To ensure our application's scalability, resource and time optimization we decided to design it as a set of microservices, each responsible for a specific portion of the overall task. With this design each service will be able to scale and be developed separately, ensuring high availability, constant non-conflicting updates and high resilience.

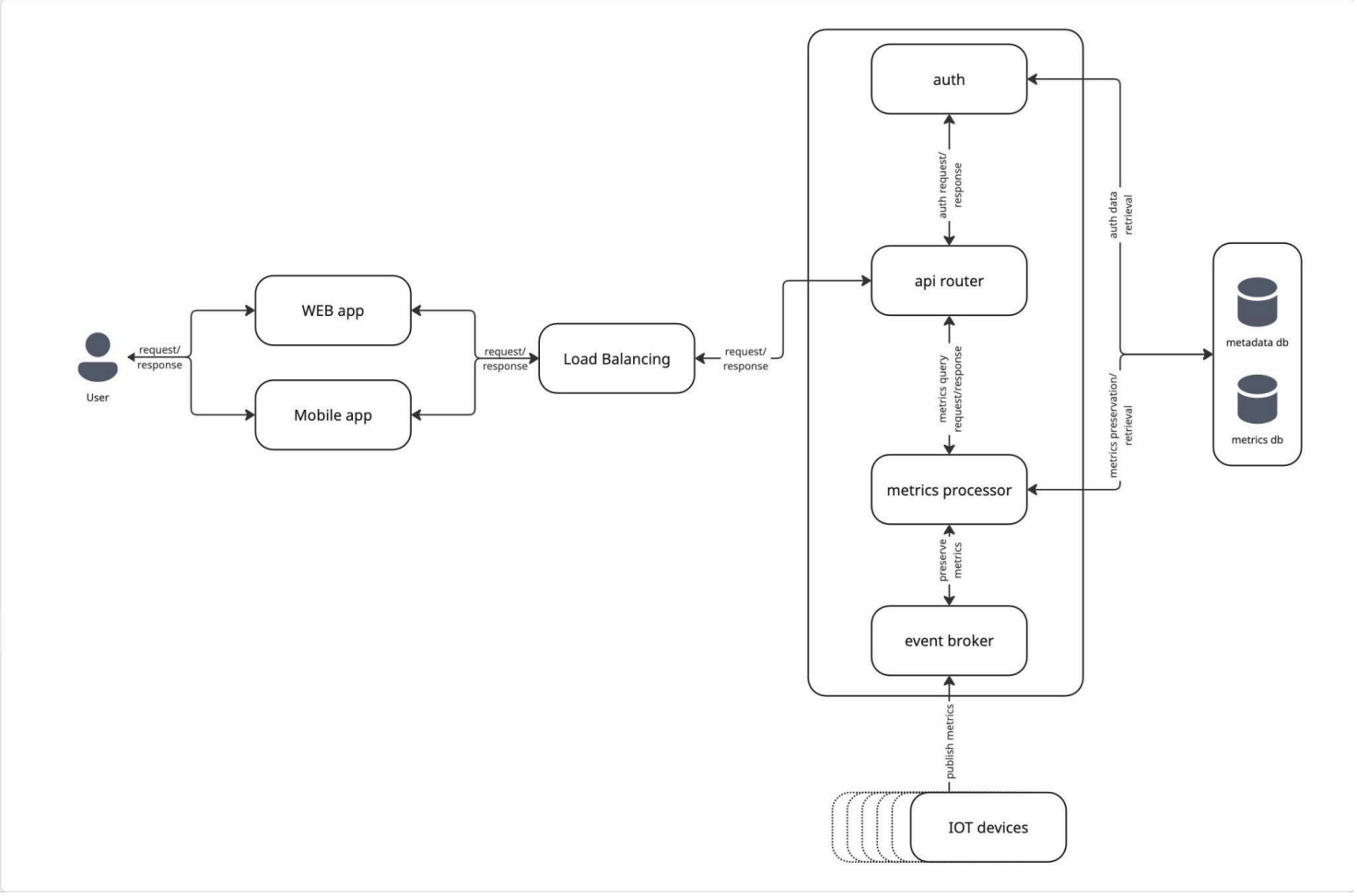
For microservices management an orchestrator will be utilized (i.e. k8s) to automate resource management, scaling and cross-service communication. Most of the services will be hosted in a private subnet, user access will be only allowed through API calls via a gateway, enhancing security and availability in case of DoS attacks. Additionally, to ensure constant service monitoring and alerting a set of internal metric collection and aggregation tools will be used.

Also 2 different types of databases were chosen to handle different aspects of working with application data - regular relational db for auth/meta data and timeseries db for storing and querying iot metrics, which will help to separate querying and dashboard building into a separate logic and enhance performance.

Detailed component description is as follows:

1. Databases
    a. metadata db (relational db) - stores user info, related device metadata, etc.
    b. metrics db (timeseries db) - for storing collected metrics
2. User App - a wrapper for calling API endpoints, rendering visuals for returned metrics
    a. Front-end for web-version
    b. Mobile App on Android and IOS
3. IOT devices - metrics-providing devices queried by the customer, main data source for dashboards
4. Load balancing - direct incoming requests to appropriate microservices, handle proxying
5. Services Orchestration (k8s) - middleware for managing microservice communication, scaling, etc.:
    a. M1 (auth) - user authentication and authorization (i.e for scoping - users must only query their devices)
    b. M2 (api) - endpoints collection for auth, querying devices and dashboard data retrieval
    c. M3 (broker, MQTT) - iot healthcheck, metrics collection, background topic subscription to known devices for timeseries data collection
    d. M4 (metrics and dashboards processor) - active/passive (some metrics are calculated before queries, some only on-demand) data storing/processing for visualization
    e. M5 (monitoring stack) - metrics collection from internal services for alerting and observability.
6. User - end consumer for visual data and device management endpoints

The schematic architecture with all components:




## 3.3 Technology stack

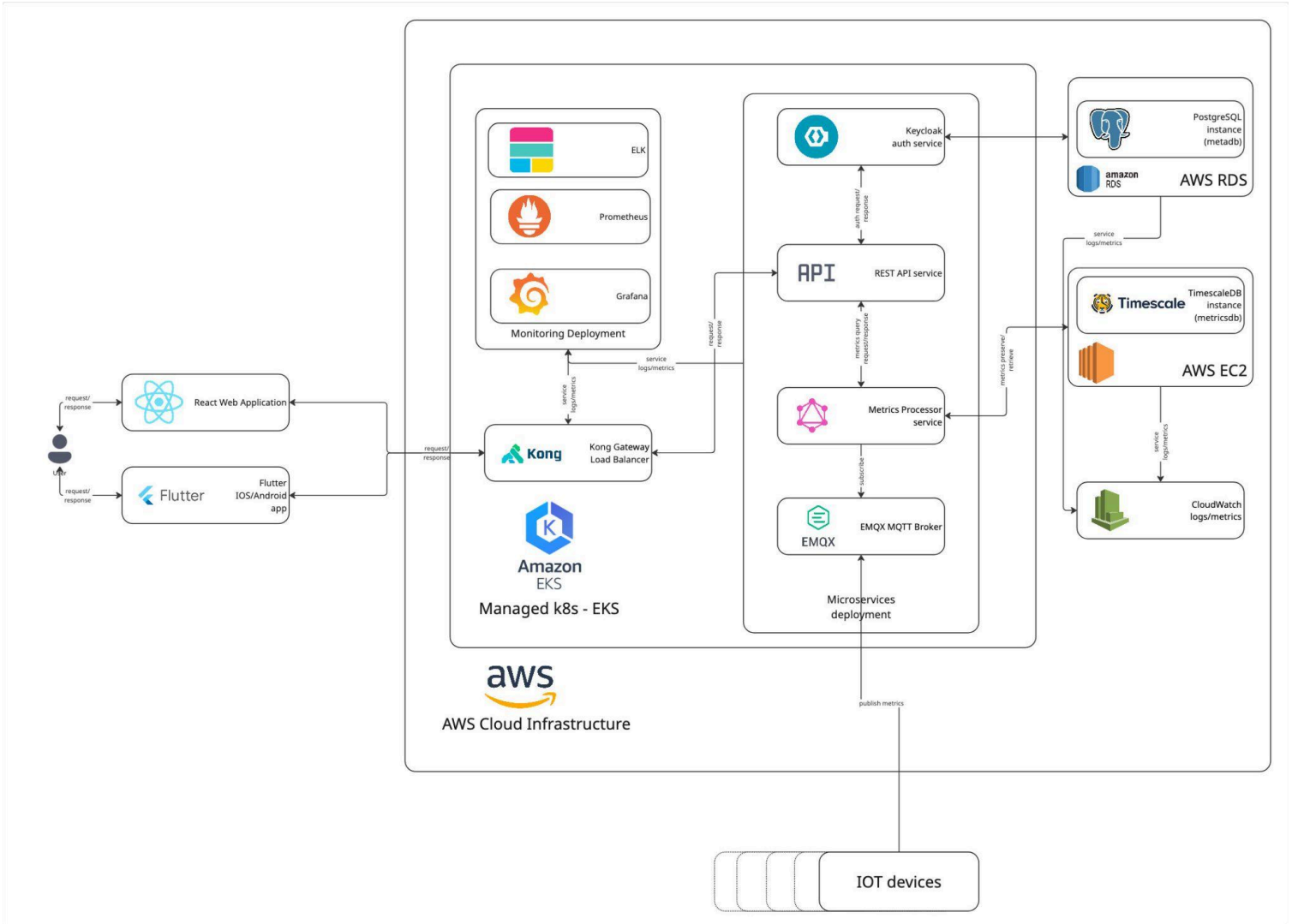
- Databases:
    a. metadata db - PostgreSQL for customers, devices and role-based access control data (deployed/configured with AWS RDS).
    b. metrics db - TimescaleDB for joining metrics with relational metadata (deployed on AWS EC2).
- UserApps:
    a. Front-end: React - providing high performance and stability, also supports backward compatibility assuring long-time support.
    b. Mobile app: Flutter is good for compiling high-performance applications, providing high development speed with hot reloads and comprehensive SDK.
- IOT devices communicate with system using MQTT protocol publishing events to broker using mTLS.
- Load balancing: Kong Gateway - is a cloud-native gateway that runs natively on Kubernetes, providing routing, balancing, rate limiting and supports OIDC and mTLS.
- Orchestration: Kubernetes for deployment, scaling, resiliency:
    a. Auth - Keycloak open-source access management solution that supports Kong OIDC.

b. API - REST for centralized queries.
c. Event broker - EMQX MQTT Broker
d. Metrics processor - service processing events from broker, normalizing and enriching them and writing them into DB. Dashboard - GraphQL is great for data-rich UI and cross-platform apps, reducing complexity.
e. Monitoring - Prometheus and Grafana for kubernetes metrics (healthchecks) and ELK for logs from services. For DBs deployed outside of k8s, CloudWatch can be used for this purpose.

- The system will be cloud based using Amazon Web Services(AWS) because it has Elastic Kubernetes Service(EKS) that makes it easier to deploy, manage and scale containerized applications, also  it has the widest managed service catalog for IoT compatible systems.

## 3.4 Architecture design

The architecture design with mapped components and chosen software:



## 3.5 Short ADRs (Architectural Decision Record)

**ADR for Databases**

We chose PostgreSQL for storing metadata (user info, device data, etc.) due to its reliability, relational nature, and ease of use with AWS RDS. For time-series data, we opted for TimescaleDB to efficiently handle large volumes of IoT metrics, providing better performance for time-dependent queries.

**ADR for UserApps**

React is one of the most commonly used frameworks for designing modern web-apps. It provides a variety of libs that are essential for Data consistency & Integrity with different services, Scalability and Flexibility of User applications. And also it provides great tools to create really User friendly application. React works very well with other architecture choices, like Rest API, Databases, k8s.

Flutter is used for the mobile app, ensuring high development speed and native performance across both iOS and Android platforms.

**ADR for Load Balancing**

Kong Gateway is used for load balancing, routing, rate-limiting, and mTLS support, running natively on Kubernetes. It ensures scalability, secure API management, and smooth traffic distribution across microservices.

**ADR for Orchestration and AWS**

Kubernetes (EKS on AWS) is chosen for orchestrating microservices to automate scaling, resource management, and cross-service communication. It enhances system reliability and allows easy handling of failures and auto-scaling.

By deploying applications across multiple AZs (availability zones), while using AWS, we enhance availability of the service.

**ADR for main components**

Keycloak provides centralized authentication and authorization through role-based access control. This means each user can only view and manage their own devices. It also integrates seamlessly with the Kong Gateway fitting perfectly into the system.

REST is a strong fit for this IoT system because it provides simple, predictable, and widely adopted HTTP interfaces that work seamlessly with web and mobile clients, and API gateways.

EMQX MQTT broker supports IoT standards and integrates smoothly to Kubernetes. It ensures normalization and enrichment of data optimizing query time for GraphQL dashboards.

Prometheus, Grafana and ELK enabling comprehensive observability, monitoring, logging and alerting.

The result system is a resilient, extensible platform that takes real-time data from multiple IoT devices, keeps it safe and organized, and turns it into fast comprehensive dashboards for people to use. System built in small independent parts, providing convenient scalability. Security and reliability provided via strong access controls, high availability and backups.